DAY 2
Build a RAG PDF Assistant

Step-by-step Google Colab lab | Retrieval-Augmented Generation

Level: Beginner **Platform:** Google Colab **API key:** Not required

Learn to load a PDF, split text into chunks, create embeddings, search them with ChromaDB, and generate an answer using retrieved document passages.

How to follow: Open the companion .ipynb in Google Colab. For each numbered cell below, read the explanation, copy or run the matching cell, and check its output before moving on.

Project workflow: PDF → text → chunks → embeddings → vector database → relevant passages → answer.

Note: The sample company policies are fictional. The first model download requires an internet connection. Free Colab runtimes can occasionally reset.

Code readability

Every code cell starts on a new page, with large monospaced text and generous line spacing. The complete code is also provided in the notebook so students do not need to retype it.

Day 2 - RAG Practical

Agamya Cyber Tech | Google Colab | Beginner-friendly, no API key. Run cells in order.

1. Open Google Colab

1. Visit https://colab.research.google.com. Sign in with your Google account. 2. Click New notebook. 3. Rename it Day2_RAG_Practical. 4. Use Runtime > Change runtime type > CPU (a free GPU may also work). 5. Run every code cell using the play button or Shift+Enter.

Expected result: A blank notebook with a Python code cell. Colab installs reset when the session restarts, so rerun the setup cell when necessary.

Cell 1 - Install the free libraries

```
%pip -q install -U pypdf fpdf2 sentence-transformers chromadb \
    langchain-community langchain-text-splitters \
    langchain-huggingface langchain-chroma transformers
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

What this does: pypdf reads PDFs; fpdf2 makes a practice PDF; Sentence Transformers creates embeddings; ChromaDB stores/searches vectors; LangChain connects document loading, splitting, embeddings and retrieval; Transformers runs our free local model.

Cell 2 - Create a sample company handbook PDF

Run this cell first even if you want to upload your own PDF later. It creates a safe and predictable document that allows everyone to test the same questions.

```
from fpdf import FPDF

policies = [
    ("Annual Leave Policy",
     "Employees receive 20 days of paid annual leave each year. "
     "Leave resets every January. Apply at least 3 days in advance."),
    ("Remote Work Policy",
     "Employees may work from home up to 2 days per week "
     "with manager approval. The arrangement is reviewed every 6 months."),
    ("Expense Policy",
     "Travel expenses above Rs 5,000 require written finance "
     "approval before the booking is made."),
    ("Code of Conduct",
     "Every employee must complete anti-harassment training "
     "once each year."),
]
pdf = FPDF()
pdf.add_page()
pdf.set_font("Helvetica", "B", 17)
pdf.cell(0, 12, "Sample Company Handbook", new_x="LMARGIN", new_y="NEXT")
for heading, content in policies:
    pdf.ln(5)
    pdf.set_font("Helvetica", "B", 12)
    pdf.multi_cell(0, 8, heading)
    pdf.set_font("Helvetica", "", 11)
    pdf.multi_cell(0, 7, content)
pdf.output("company_handbook.pdf")
print("Created company_handbook.pdf")
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

Expected result: The output says Created company_handbook.pdf. The file appears in the Files panel on the left of Colab.

Cell 3 - Load the PDF and inspect its pages

```
from langchain_community.document_loaders import PyPDFLoader

pdf_path = "company_handbook.pdf"
docs = PyPDFLoader(pdf_path).load()
print("Pages loaded:", len(docs))
print("First page text (preview):")
print(docs[0].page_content[:800])
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

Expected result: At least one page loaded, displaying annual leave, remote work, expenses and code of conduct. Notice the PDF text is now available to Python.

Cell 4 - Divide text into smaller chunks

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=300, chunk_overlap=40
)
chunks = splitter.split_documents(docs)
print("Number of chunks:", len(chunks))
for i, chunk in enumerate(chunks[:4], start=1):
    print(f"\nCHUNK {i}: {chunk.page_content[:250]}")
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

Observe: Chunk size limits how much text is searched at once; overlap repeats a little text at the boundary to preserve meaning. Try changing chunk_size=300 to 150 and rerun the cell. Smaller chunks usually create more pieces.

Cell 5 - Convert chunks into embeddings and store in ChromaDB

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
vector_db = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    collection_name="day2_handbook"
)
print("Stored chunks:", vector_db._collection.count())
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

Expected result: A number greater than zero. The first execution downloads the free embedding model; subsequent executions in the same runtime are usually faster. Embeddings are numeric representations of meaning, not generated answers.

Cell 6 - Ask ChromaDB to find relevant information

```
question = "How often can staff work from home?"
relevant = vector_db.similarity_search(question, k=2)
for rank, item in enumerate(relevant, start=1):
    page = item.metadata.get("page", 0) + 1
    print(f"MATCH {rank} | PDF page {page}")
    print(item.page_content)
    print("-" * 50)
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

Expected result: A chunk mentioning up to 2 days per week with manager approval should appear among the matches. Retrieval is the most important check: if the correct text is missing, the LLM cannot reliably use it.

2. Generate an answer using a free language model

The retrieval code above already proves semantic search works. Now add a lightweight language model to turn the retrieved text into an answer. No Gemini, OpenAI or other paid API key is necessary. The model is downloaded into the Colab runtime; free CPU inference may be slow.

Cell 7 - Load the open-source text-generation model

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

model_id = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_id)
language_model = AutoModelForSeq2SeqLM.from_pretrained(model_id)
language_model.eval()
print("Ready:", model_id)
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

Expected result: Ready: google/flan-t5-small. This model is intentionally small so that students can run it in a basic Colab session; answer quality will not match a large commercial model.

Cell 8 - Build the complete RAG question-answering function

```
def ask_handbook(question):
    matches = vector_db.similarity_search(question, k=2)
    context = "\n".join(item.page_content for item in matches)
    prompt = (
        "Answer the question using only the context below. "
        "If the answer is not stated, say 'Not found in the document'.\n"
        f"Context:\n{context}\nQuestion: {question}\nAnswer:"
    )
    inputs = tokenizer(prompt, return_tensors="pt",
                       max_length=450, truncation=True)
    with torch.no_grad():
        output = language_model.generate(**inputs, max_new_tokens=70)
    answer = tokenizer.decode(output[0], skip_special_tokens=True)
    print("QUESTION:", question)
    print("ANSWER:", answer)
    print("\nSOURCES (check these against the answer):")
    for item in matches:
        print(f"PDF page {item.metadata.get('page', 0) + 1}: "
              f"{item.page_content[:180]}")
    return answer

ask_handbook("How many paid annual leave days do employees get?")
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

Expected result: An answer indicating 20 days, plus the source excerpts. Because the model is small, wording or accuracy can vary. Always verify against the retrieved source.

Cell 9 - Ask more questions

```
ask_handbook("How many days per week is remote work allowed?")
ask_handbook("When does annual leave reset?")
ask_handbook("When must finance approve a travel expense?")
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

Correct source facts: Remote work up to 2 days/week with manager approval; leave resets every January; travel expenses above Rs 5,000 require written approval before booking.

3. Optional - Use your own PDF

Only use a PDF you are allowed to upload. Do not upload passwords, customer data, confidential company records or personal information into a workshop environment. Text-based PDFs work best. Scanned/image-only PDFs require OCR, which is not included in this lab.

Cell 10 - Upload a PDF

```
from google.colab import files
uploaded = files.upload()
my_pdf = next(iter(uploaded))
print("Uploaded:", my_pdf)
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

PRACTICAL STEP 11 / 11

Cell 11 - Rebuild the index for the uploaded PDF

```
my_docs = PyPDFLoader(my_pdf).load()
my_chunks = splitter.split_documents(my_docs)
vector_db = Chroma.from_documents(
    documents=my_chunks,
    embedding=embeddings,
    collection_name="uploaded_pdf"
)
print("Indexed", len(my_chunks), "chunks from", len(my_docs), "pages")
ask_handbook("What is the main topic of this document?")
```

Run this cell in Colab using Shift + Enter. Review the printed output before going to the next step.

Important: Ask questions related to your uploaded document. The first question may be too broad for a tiny model; try asking for a specific fact that appears in the PDF. Each run creates a new in-memory collection; your index is lost when the Colab session resets.

4. Understand what you built

PDF loading: Extracts readable text. Chunking: Breaks it into manageable pieces. Embeddings: Encode the meaning of each piece. ChromaDB: Stores those vectors and searches for similar ones. Retrieval: Selects matching chunks. Augmentation: Adds chunks to the model prompt. Generation: Produces a natural-language answer. LangChain: Provides the loader, splitter, embeddings and vector store interfaces used in this workflow.

5. Student activity and submission

Task A: Find the remote-work policy using the retriever. Show the matching source text. Task B: Ask the RAG system about leave entitlement and expense approval. Task C: Change chunk_size to 150, then rerun the chunking and indexing cells and record how the number of chunks changes. Task D: Ask a question not answered in the document (for example, "What is the CEO salary?"). Explain why the answer should not be trusted if unsupported.

Submit: From Colab select File > Download > Download .ipynb. Submit the notebook with output for at least three questions, the retrieved source for each, and a 3-5 sentence reflection about RAG versus a normal LLM.

6. Common errors and fixes

7. Quick review

1. Why do we use RAG rather than asking only the LLM? 2. What is an embedding? 3. How is semantic search different from keyword search? 4. Why do we split large PDFs into chunks? 5. What role does ChromaDB play?

Instructor note: This practical follows the Day 2 Student Handbook sequence, including the fictional leave, remote work, expense and conduct examples. Current LangChain package imports are used rather than the older imports shown in the slides. Model outputs are illustrative, not guaranteed. The environment needs internet on first model download.